

Descompunerea Tucker a Tensorilor

Documentație matematică

Proiect Calcul Numeric

May 8, 2026

Contents

1	Descrierea matematică a metodelor	3
1.1	Descompunerea după valori singulare (SVD)	3
1.1.1	Definiție și existență	3
1.1.2	SVD redus	3
1.1.3	Proprietatea de aproximare optimă	4
1.2	Noțiuni tensoriale	4
1.2.1	Matricizarea (unfolding)	4
1.2.2	Produsul mod- n	5
1.3	Descompunerea Tucker și HOSVD	5
1.3.1	Definiția Tucker	5
1.3.2	Algoritmul HOSVD	5
1.3.3	Eroarea de aproximare	5
1.3.4	Factorul de compresie	6
2	Motivarea funcțiilor implementate	6
2.1	Funcții auxiliare	6
2.1.1	<code>norma_frobenius</code>	6
2.1.2	<code>norma_extra_diag</code>	7
2.2	Factorizarea QR	7
2.2.1	<code>QR_Gram_Schmidt</code>	7
2.2.2	<code>QR_Householder</code>	7
2.3	Reducerea la formă tridiagonală	7
2.3.1	<code>Tridiag_Householder</code>	7
2.4	Calculul valorilor proprii	8
2.4.1	<code>QR_iteration</code>	8
2.5	Descompunerea SVD	8
2.5.1	SVD	8
2.5.2	SVD redus	9
2.6	Funcții tensoriale	9
2.6.1	<code>permutare</code> și <code>permutare_inv</code>	9
2.6.2	<code>matricize</code>	9
2.6.3	<code>mode_n_product</code>	9
2.6.4	HOSVD	9
2.6.5	<code>reconstruct</code>	10

2.6.6	tucker_error	10
2.6.7	margineteoretica	10

1 Descrierea matematică a metodelor

1.1 Descompunerea după valori singulare (SVD)

1.1.1 Definiție și existență

Definiție 1.1. Fie $A \in \mathbb{R}^{m \times n}$. Descompunerea după valori singulare (SVD) a lui A este factorizarea

$$A = U\Sigma V^T$$

unde:

- $U \in \mathbb{R}^{m \times m}$ este ortogonală: $U^T U = U U^T = I_m$
- $V \in \mathbb{R}^{n \times n}$ este ortogonală: $V^T V = V V^T = I_n$
- $\Sigma \in \mathbb{R}^{m \times n}$ este pseudo-diagonală cu intrările $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0$, unde $p = \min(m, n)$, numite **valori singulare** ale lui A

Teoremă 1.1 (Existența SVD). Orice matrice $A \in \mathbb{R}^{m \times n}$ admite o descompunere după valori singulare.

Proof. Matricea $A^T A \in \mathbb{R}^{n \times n}$ este simetrică și pozitiv semidefinită, deci prin teorema spectrală admite o diagonalizare ortogonală:

$$A^T A = V \Lambda V^T$$

unde V este ortogonală și $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ cu $\lambda_i \geq 0$. Definim valorile singulare $\sigma_i = \sqrt{\lambda_i}$.

Pentru $\sigma_i > 0$, definim vectorii singulari stângi:

$$u_i = \frac{A v_i}{\sigma_i}, \quad i = 1, \dots, r$$

unde $r = \text{rang}(A)$. Aceștia sunt ortonormați:

$$\langle u_i, u_j \rangle = \frac{v_i^T A^T A v_j}{\sigma_i \sigma_j} = \frac{\sigma_j^2 \langle v_i, v_j \rangle}{\sigma_i \sigma_j} = \delta_{ij}$$

Completăm $\{u_1, \dots, u_r\}$ la o bază ortonormată a lui $\mathbb{R}^m$ prin procesul Gram-Schmidt. Matricea $U = [u_1 \mid \dots \mid u_m]$ este ortogonală și se verifică $A = U\Sigma V^T$. $\square$

1.1.2 SVD redus

Definiție 1.2. SVD-ul redus (thin SVD) de rang r al matricei A este

$$A \approx U_r \Sigma_r V_r^T$$

unde $U_r \in \mathbb{R}^{m \times r}$, $\Sigma_r \in \mathbb{R}^{r \times r}$, $V_r \in \mathbb{R}^{n \times r}$ conțin doar primele r coloane/valori singulare.

1.1.3 Proprietatea de aproximare optimă

Teoremă 1.2 (Eckart–Young–Mirsky). Fie $A \in \mathbb{R}^{m \times n}$ cu $\text{rang}(A) = \rho$ și SVD-ul $A = U\Sigma V^T$. Atunci, pentru orice $r \leq \rho$,

$$\min_{\text{rang}(B) \leq r} \|A - B\|_F = \|A - A_r\|_F = \sqrt{\sigma_{r+1}^2 + \sigma_{r+2}^2 + \cdots + \sigma_\rho^2}$$

unde $A_r = U_r \Sigma_r V_r^T$ este trunchierea SVD la rang r .

Această teoremă justifică de ce trunchierea SVD este metoda optimă de aproximare de rang scăzut: nu există altă matrice de rang $\leq r$ mai apropiată de A în norma Frobenius.

1.2 Noțiuni tensoriale

Definiție 1.3. Un **tensor de ordinul N** este un element $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$. Elementele sale sunt indexate $a_{i_1 i_2 \cdots i_N}$.

Observație 1.1. Scalarii, vectorii și matricele sunt cazuri particulare: tensori de ordinul 0, 1, respectiv 2. O imagine color de dimensiuni $H \times W$ este un tensor de ordinul 3 cu shape $H \times W \times 3$, unde a treia dimensiune corespunde canalelor RGB.

1.2.1 Matricizarea (unfolding)

Definiție 1.4. **Matricizarea pe modul n** a tensorului $\mathcal{A} \in \mathbb{R}^{I_1 \times \cdots \times I_N}$ este matricea $A_{(n)} \in \mathbb{R}^{I_n \times (I_1 \cdots I_{n-1} I_{n+1} \cdots I_N)}$ obținută plasând modul n pe linii și aplatizând toate celelalte moduri pe coloane.

Formal, elementul $(i_1, \dots, i_N)$ din $\mathcal{A}$ mapează la poziția (i_n, j) din $A_{(n)}$, unde

$$j = 1 + \sum_{\substack{k=1 \\ k \neq n}}^N (i_k - 1) \prod_{\substack{m=1 \\ m \neq n}}^{k-1} I_m$$

Exemplu numeric. Fie $\mathcal{A} \in \mathbb{R}^{3 \times 4 \times 2}$ cu $\mathcal{A}[:, :, 1] = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{pmatrix}$ și $\mathcal{A}[:, :, 2] =$

$$\begin{pmatrix} 13 & 14 & 15 & 16 \\ 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 \end{pmatrix}.$$

Matricizările sunt:

$$A_{(1)} = \begin{pmatrix} 1 & 2 & 3 & 4 & 13 & 14 & 15 & 16 \\ 5 & 6 & 7 & 8 & 17 & 18 & 19 & 20 \\ 9 & 10 & 11 & 12 & 21 & 22 & 23 & 24 \end{pmatrix} \in \mathbb{R}^{3 \times 8}$$

$$A_{(2)} = \begin{pmatrix} 1 & 5 & 9 & 13 & 17 & 21 \\ 2 & 6 & 10 & 14 & 18 & 22 \\ 3 & 7 & 11 & 15 & 19 & 23 \\ 4 & 8 & 12 & 16 & 20 & 24 \end{pmatrix} \in \mathbb{R}^{4 \times 6}$$

$$A_{(3)} = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & & & & & & & & \\ 13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 22 \\ 23 & 24 & & & & & & & & \end{pmatrix} \in \mathbb{R}^{2 \times 12}$$

1.2.2 Produsul mod- n

Definiție 1.5. Produsul mod- n al tensorului $\mathcal{A} \in \mathbb{R}^{I_1 \times \dots \times I_N}$ cu matricea $M \in \mathbb{R}^{J \times I_n}$ este tensorul $\mathcal{B} = \mathcal{A} \times_n M \in \mathbb{R}^{I_1 \times \dots \times I_{n-1} \times J \times I_{n+1} \times \dots \times I_N}$ definit prin

$$b_{i_1 \dots i_{n-1} j i_{n+1} \dots i_N} = \sum_{i_n=1}^{I_n} a_{i_1 \dots i_N} \cdot m_{j, i_n}$$

Proprietate 1.1 (Relația cu matricizarea).

$$(\mathcal{A} \times_n M)_{(n)} = M \cdot A_{(n)}$$

Această proprietate este fundamentală pentru implementare: reduce produsul mod- n la o înmulțire matriceală obișnuită aplicată pe matricizarea tensorului.

Proprietate 1.2 (Comutativitate pe moduri diferite). Pentru $m \neq n$:

$$(\mathcal{A} \times_m M) \times_n N = (\mathcal{A} \times_n N) \times_m M$$

1.3 Descompunerea Tucker și HOSVD

1.3.1 Definiția Tucker

Definiție 1.6. Descompunerea Tucker de multi-rang (R_1, R_2, R_3) a tensorului $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$ este factorizarea

$$\mathcal{A} = \mathcal{G} \times_1 U^{(1)} \times_2 U^{(2)} \times_3 U^{(3)}$$

unde:

- $\mathcal{G} \in \mathbb{R}^{R_1 \times R_2 \times R_3}$ este **tensorul de bază** (core tensor)
- $U^{(n)} \in \mathbb{R}^{I_n \times R_n}$ sunt **matricele factoriale**, cu coloane ortonormate: $(U^{(n)})^T U^{(n)} = I_{R_n}$

Alegând $R_n < I_n$ pentru cel puțin un mod, obținem o **aproximare comprimată** a tensorului original.

1.3.2 Algoritmul HOSVD

Higher-Order SVD (HOSVD), introdus de De Lathauwer et al. [3], calculează o descompunere Tucker prin SVD-uri independente pe fiecare matricizare.

Reconstrucția tensorului aproximat se face prin:

$$\tilde{\mathcal{A}} = \mathcal{G} \times_1 \hat{U}^{(1)} \times_2 \hat{U}^{(2)} \times_3 \hat{U}^{(3)}$$

1.3.3 Eroarea de aproximare

Teoremă 1.3 (Marginea erorii HOSVD, [2]). Fie $\tilde{\mathcal{A}}$ aproximarea Tucker de multi-rang (R_1, R_2, R_3) obținută prin HOSVD. Atunci:

$$\|\mathcal{A} - \tilde{\mathcal{A}}\|_F \leq \sqrt{\sum_{n=1}^3 \sum_{i_n=R_n+1}^{I_n} \left(\sigma_{i_n}^{(n)}\right)^2}$$

unde $\sigma_{i_n}^{(n)}$ sunt valorile singulare ale matricizării $A_{(n)}$.

Algorithm 1 HOSVD

Require: Tensor $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$, ranguri (R_1, R_2, R_3) cu $R_n \leq I_n$

Ensure: Core tensor $\mathcal{G}$, matrice factoriale $U^{(1)}, U^{(2)}, U^{(3)}$

- 1: **for** $n = 1, 2, 3$ **do**
 - 2: Calculează $A_{(n)} \leftarrow \text{matricize}(\mathcal{A}, n)$
 - 3: Calculează SVD: $A_{(n)} = U^{(n)} \Sigma^{(n)} V^{(n)T}$
 - 4: $\hat{U}^{(n)} \leftarrow$ primele R_n coloane din $U^{(n)}$
 - 5: **end for**
 - 6: $\mathcal{G} \leftarrow \mathcal{A} \times_1 (\hat{U}^{(1)})^T \times_2 (\hat{U}^{(2)})^T \times_3 (\hat{U}^{(3)})^T$ **return** $\mathcal{G}, \hat{U}^{(1)}, \hat{U}^{(2)}, \hat{U}^{(3)}$
-

Schiță de demonstrație. Notăm cu $P_n = \hat{U}^{(n)}(\hat{U}^{(n)})^T$ proiectorul ortogonal pe subspațiul generat de primele R_n coloane din $U^{(n)}$.

Eroarea de proiecție pe modul n singur este:

$$\|\mathcal{A} - \mathcal{A} \times_n P_n\|_F^2 = \sum_{i_n=R_n+1}^{I_n} (\sigma_{i_n}^{(n)})^2$$

prin teorema Eckart–Young aplicată pe matricizarea $A_{(n)}$.

Aplicând inegalitatea triunghiului pentru proiecții succesive și folosind proprietatea de comutativitate a produselor mod- n pe moduri diferite, obținem marginea sumei. $\square$

Observație 1.2. HOSVD nu furnizează aproximarea optimă de multi-rang (R_1, R_2, R_3) . Inegalitatea din teoremă este în general strictă. Aproximarea optimă se obține prin algoritmul HOOI (Higher-Order Orthogonal Iteration), care rafinează soluția HOSVD prin iterații alternante, dar nu este implementat în acest proiect.

1.3.4 Factorul de compresie

Stocarea tensorului original necesită $\prod_{n=1}^3 I_n$ numere. Stocarea descompunerii Tucker necesită:

$$\underbrace{R_1 R_2 R_3}_{\mathcal{G}} + \underbrace{I_1 R_1 + I_2 R_2 + I_3 R_3}_{U^{(1)}, U^{(2)}, U^{(3)}} \text{ numere}$$

Factorul de compresie este:

$$\text{CR} = \frac{I_1 I_2 I_3}{R_1 R_2 R_3 + I_1 R_1 + I_2 R_2 + I_3 R_3}$$

2 Motivarea funcțiilor implementate

2.1 Funcții auxiliare

2.1.1 norma_frobenius

Definiție 2.1. Norma Frobenius a unei matrice $A \in \mathbb{R}^{m \times n}$ este:

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2}$$

Norma Frobenius este echivalentă cu norma euclidiană a vectorului obținut prin aplatizarea matricei. Este aleasă în locul normei spectrale deoarece este calculabilă direct fără a recurge la valori proprii, și este extensibilă natural la tensori.

Listing 1: Implementarea normei Frobenius

```
def norma_frobenius(A):
    return np.sqrt(np.sum(A ** 2))
```

2.1.2 norma_extra_diag

Folosită ca criteriu de convergență în iterația QR. Măsoară cât de departe este o matrice de a fi diagonală — iterația QR converge când elementele în afara diagonalei sunt neglijabile:

$$\|A - \text{diag}(A)\|_F < \varepsilon$$

2.2 Factorizarea QR

2.2.1 QR_Gram_Schmidt

Procesul Gram-Schmidt clasic ortogonalizează coloanele matricei A succesiv. La pasul k , vectorul curent se proiectează pe toți vectorii deja ortonormați și componenta ortogonală devine noua coloană din Q :

$$\tilde{q}_k = a_k - \sum_{i=1}^{k-1} \langle q_i, a_k \rangle q_i, \quad q_k = \frac{\tilde{q}_k}{\|\tilde{q}_k\|}$$

Această funcție este utilizată în interiorul iterației QR pentru calculul valorilor proprii. Referință: Golub & Van Loan [1], Algoritmul 5.2.1.

2.2.2 QR_Householder

Factorizarea QR prin reflectori Householder este mai stabilă numeric decât Gram-Schmidt clasic. La pasul k , se construiește un reflector $H_k = I - 2vv^T$ cu

$$v = x - \alpha e_1, \quad \alpha = -\text{semn}(x_1) \|x\|_2$$

care anulează elementele sub-diagonale ale coloanei k .

Această funcție nu este folosită direct în SVD, dar este implementată ca alternativă mai robustă și este comparată cu Gram-Schmidt în testele numerice. Referință: Golub & Van Loan [1], Secțiunea 5.1.

2.3 Reducerea la formă tridiagonală

2.3.1 Tridiag_Householder

Orice matrice simetrică A poate fi redusă la formă tridiagonală prin transformări de similaritate ortogonale:

$$Q^T A Q = T, \quad T \text{ tridiagonală}$$

La pasul k , se alege reflectorul Householder care anulează elementele $a_{k+2,k}, a_{k+3,k}, \dots, a_{n,k}$ din coloana k . Deoarece transformarea se aplică bilateral (HTH), simetria este păstrată.

Reducerea la tridiagonal este necesară deoarece **iterația QR converge mult mai rapid pe matrice tridiagonale** — costul per iterație scade de la $O(n^3)$ la $O(n)$. Referință: Golub & Van Loan [1], Algoritmul 8.3.1.

2.4 Calculul valorilor proprii

2.4.1 QR_iteration

Iteratia QR fara shift porneste de la $T_0 = Q_0^T A Q_0$ (matricea tridiagonalizata) si aplica repetat:

$$T_k = Q_k R_k \Rightarrow T_{k+1} = R_k Q_k$$

Secventa T_k converge catre o matrice diagonala ale carei intrari sunt valorile proprii ale lui A , iar matricea acumulata $V = Q_0 Q_1 Q_2 \dots$ contine vectorii proprii corespunzatori.

Accelerare prin shift. Convergenta iteratiei QR poate fi lenta cand valorile proprii sunt apropiate. O tehnica standard de accelerare este **shift-ul spectral**: la fiecare pas k se scade un scalar μ_k (shift-ul) inainte de factorizare si se readauga dupa:

$$T_k - \mu_k I = Q_k R_k \Rightarrow T_{k+1} = R_k Q_k + \mu_k I$$

Matricea T_{k+1} are aceleasi valori proprii ca T_k (shift-ul nu le modifica), dar convergenta este semnificativ mai rapida.

In implementarea noastra se foloseste **shift-ul simplu Rayleigh**:

$$\mu_k = T_k[n-1, n-1]$$

adica elementul din coltul dreapta-jos al matricei curente. Aceasta alegere este motivata de faptul ca, pe masura ce iteratia converge, $T_k[n-1, n-1]$ se apropie de cea mai mica valoare proprie in modul — exact valoarea catre care converge ultima linie. Shiftand cu ea, acceleram convergenta ultimei valori proprii de la liniara la cubica (in cazul shift-ului Wilkinson complet).

Criteriul de oprire combina precizia dorita cu un numar maxim de iteratii pentru a garanta terminarea:

$$\|T_k - \text{diag}(T_k)\|_F < \varepsilon \quad \text{sau} \quad k \geq k_{\max}$$

implementat prin `norma_extra_diag` si parametrul `max_iter=1000`.

Referinta: Golub & Van Loan [1], Sectiunea 8.3.

2.5 Descompunerea SVD

2.5.1 SVD

Strategia de calcul are trei pasi:

Pasul 1 — Valorile singulare. Se observă că dacă $A = U \Sigma V^T$ atunci $A^T A = V \Sigma^T \Sigma V^T$, deci valorile proprii ale lui $A^T A$ sunt σ_i^2 . Se calculează:

$$\lambda_i = \text{val. proprii ale } A^T A \Rightarrow \sigma_i = \sqrt{\lambda_i}$$

prin `Tridiag_Householder` urmat de `QR_iteration`.

Pasul 2 — Vectorii singulari drepti V . Vectorii proprii ai lui $A^T A$ calculați în Pasul 1 sunt coloanele lui V .

Pasul 3 — Vectorii singulari stângi U . Pentru $\sigma_i > 0$:

$$u_i = \frac{A v_i}{\sigma_i}$$

Pentru valorile singulare nule (rang deficient), spațiul nul este completat prin Gram-Schmidt aplicat pe vectorii bazei canonice.

Observație 2.1. Abordarea prin $A^T A$ este corectă matematic, dar poate introduce erori numerice pentru matrici prost condiționate, deoarece numărul de condiție se ridică la pătrat. O alternativă mai stabilă este bidiagonalizarea directă (Golub-Reinsch), dar implementarea acesteia depășește scopul acestui proiect.

2.5.2 SVD_redus

SVD-ul redus de rang r returnează doar primele r coloane din U , primele r valori singulare și primele r linii din V^T :

$$A \approx U_r \Sigma_r V_r^T$$

Utilizare în HOSVD: pentru fiecare matricizare $A_{(n)}$, avem nevoie doar de primele R_n coloane din $U^{(n)}$ — restul sunt aruncate. SVD-ul redus evită calculul și stocarea coloanelor inutile.

2.6 Funcții tensoriale

2.6.1 permutare și permutare_inv

`permutare(N, mode)` construiește permutarea $[n, 0, 1, \dots, n-1, n+1, \dots, N-1]$ care mută modul n pe prima poziție. Aceasta este necesară pentru a aduce modul dorit pe prima axă înainte de `reshape`, astfel încât numpy să-l trateze corect ca "linii" ale matricei.

`permutare_inv` construiește inversa acestei permutări, necesară pentru a readuce modul pe poziția originală după `reshape`.

2.6.2 matricize

Implementează Definiția matricizării în trei pași:

1. Permutare axe: modul n devine prima dimensiune
2. Calculul numărului de coloane: $\prod_{k \neq n} I_k$
3. Aplatizare prin `reshape(I_n, -1)`

2.6.3 mode_n_product

Implementează Proprietatea 1.1: $(\mathcal{A} \times_n M)_{(n)} = M \cdot A_{(n)}$

Pașii:

1. Matricizare pe modul n : $A_{(n)} \leftarrow \text{matricize}(\mathcal{A}, n)$
2. Înmulțire matriceală: $B_{(n)} \leftarrow M \cdot A_{(n)}$
3. Refacere tensor: `reshape` la forma corectă + permutare inversă

2.6.4 HOSVD

Implementează direct algoritmul HOSVD descris în Secțiunea 1.3.2. Folosește `SVD_redus` în locul lui `SVD` complet pentru eficiență: calculează direct cele R_n coloane necesare fără a construi întreaga matrice U .

2.6.5 reconstruct

Inversul compresiei Tucker:

$$\tilde{\mathcal{A}} = \mathcal{G} \times_1 \hat{U}^{(1)} \times_2 \hat{U}^{(2)} \times_3 \hat{U}^{(3)}$$

Dimensiunile cresc de la (R_1, R_2, R_3) înapoi la (I_1, I_2, I_3) .

2.6.6 tucker_error

Calculează $\|\mathcal{A} - \tilde{\mathcal{A}}\|_F$ apelând `reconstruct` și aplicând `norma_frobenius` pe diferența aplatizată.

2.6.7 margine_teoretica

Calculează membrul drept din inegalitatea de eroare HOSVD fără a reconstitui tensorul. Complexitate: $O(N \cdot \max_n(I_n \cdot \prod_{k \neq n} I_k))$ — semnificativ mai ieftin decât reconstrucția completă pentru tensori mari.

References

- [1] G.H. Golub, C.F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.
- [2] T.G. Kolda, B.W. Bader, *Tensor Decompositions and Applications*, SIAM Review, 51(3):455–500, 2009.
- [3] L. De Lathauwer, B. De Moor, J. Vandewalle, *A Multilinear Singular Value Decomposition*, SIAM Journal on Matrix Analysis and Applications, 21(4):1253–1278, 2000.